

# Architecture

A guide to how **AURA** — **Ashesi University Resource Allocation** is structured: the folder layout and *why* it's shaped that way, what every package/module contains, how they depend on each other, and how a request and its data flow through the system. (Identity & naming live in [BRAND.md](#).)

Read this top-to-bottom and you should be able to find any piece of behaviour and follow it from HTTP request → service → database → response.

---

## 1. What style is this? (flat? hexagonal?)

Neither flat nor strict hexagonal. It is a **modular monolith** with **layered, bounded-context modules** over a shared **platform (infrastructure) layer**, and it borrows the one genuinely useful idea from hexagonal/ports-and-adapters: **external concerns sit behind interfaces** so they can be swapped.

Concretely:

- **One deployable API binary** (`cmd/api`) — not microservices. At this scale (one university, ~100 concurrent users) that's the right call; the module boundaries below are the *future* service seams.
- **Bounded-context modules** under `internal/`: `iam`, `catalogue`, `scheduling`, `availability`, `bookings`, `notifications`, `reporting`. Each module owns its domain and is internally **layered**:

HTTP handler (transport) → service (domain logic) (business rules) → data access (sqlc/pgx) (typed queries) →

- **Platform layer** (`internal/platform/`) is the “adapters” side: database pool, auth primitives, RBAC, HTTP plumbing, mailer, config, audit, metrics, logging. Modules depend on the platform; the platform never depends on a module.
- **Ports (interfaces) for swappable concerns**: `mailer.Mailer`, `auth.TokenSigner`, `bookings.Notifier`, `iam.Mailer`, `audit.Inserter`. Implementations are wired in `cmd/api` (composition root). This is what lets, e.g., the log-mailer become SMTP, or the in-process notifier become River, without touching domain code.

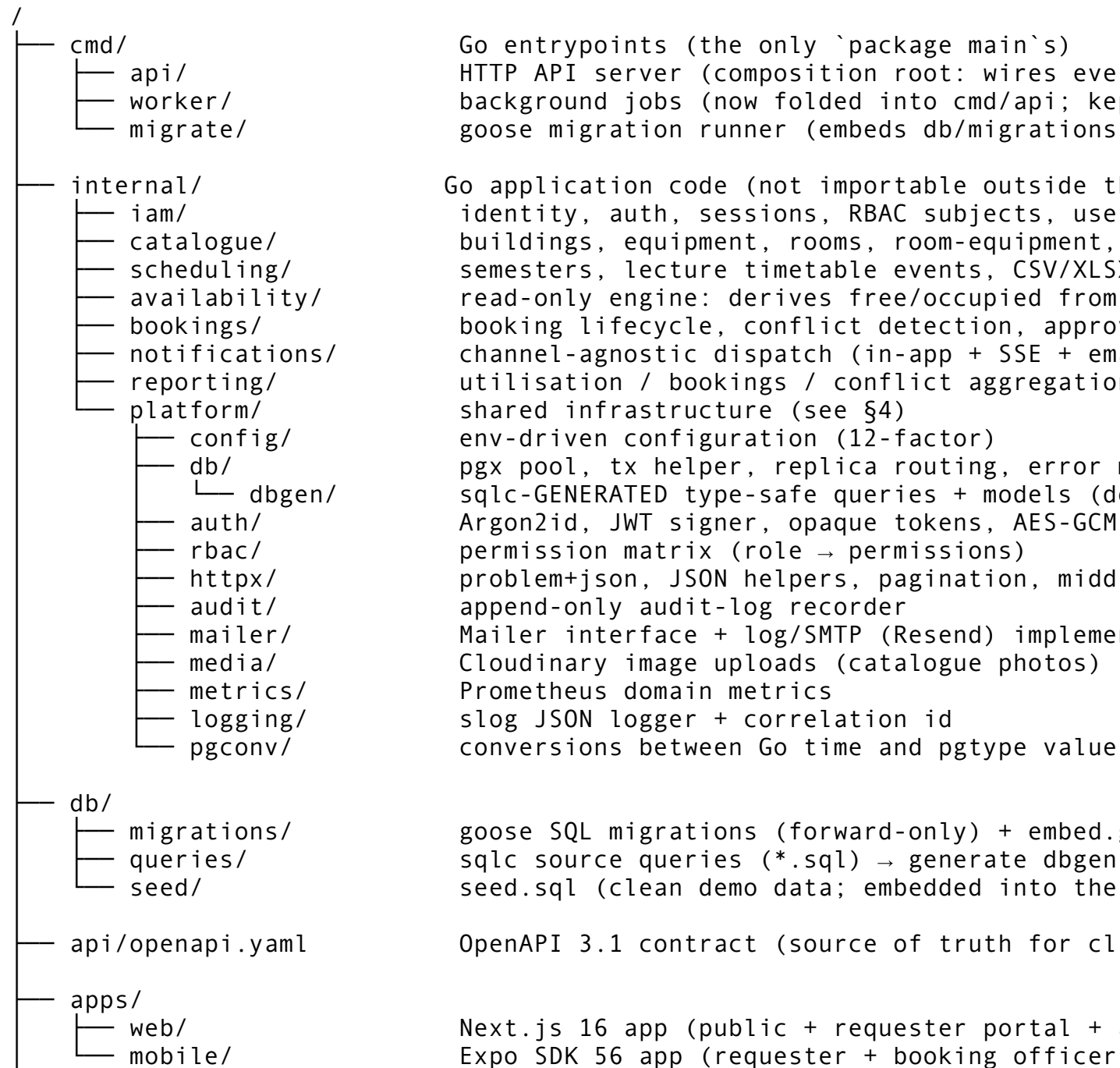
So: **module-per-bounded-context, layered inside, ports for the outside world**. Not flat (logic isn't dumped in one package); not full hexagonal (we don't abstract the database behind a repository interface — *sqlc* is the typed data layer, and the DB's constraints are deliberately part of the domain guarantees).

The **web app** is a separate axis: a Next.js App Router application organised by **route groups** + **feature folders**, consuming the API through a generated typed client.

---

## 2. Repository layout (monorepo)

Two toolchains in one repo: a **Go module** at the root (the backend) and a **pnpm + Turborepo workspace** (web, mobile, shared TS packages).



|                                    |                                              |
|------------------------------------|----------------------------------------------|
| — packages/                        | shared TS workspace packages                 |
| — api-client/                      | openapi-fetch client + generated schema.gen. |
| — schemas/                         | shared zod schemas + enums + error codes     |
| — tokens/                          | framework-neutral brand palette + dark-tint  |
| — ui/                              | Tailwind v4 tokens, shadcn-style components, |
| — config/                          | shared eslint / tsconfig                     |
| — deploy/                          |                                              |
| — compose/                         | local docker-compose stack + Dockerfile.api  |
| — render/                          | Dockerfile (one image: api+worker+migrate; R |
| — helm/                            | Kubernetes chart (alternative to Render)     |
| — terraform/                       | IaC for the K8s target                       |
| — docs/                            |                                              |
| — decisions/                       | ADRs (numbered; record non-obvious choices)  |
| — runbooks/                        | operational runbooks (incl. Render + K8s)    |
| — render.yaml                      | Render Blueprint (chosen deploy path)        |
| — go.mod / sqlc.yaml               | Go module + sqlc config                      |
| — pnpm-workspace.yaml / turbo.json | JS workspace                                 |

**The single most important design rule** (carried from the spec): *lecture occupancy and booking occupancy are stored separately*. Lectures are recurring `timetable_events` scoped to a semester; ad-hoc reservations are `bookings`. Availability is **computed** from both (plus maintenance). Replacing a timetable never touches bookings; clearing bookings never deletes lectures.

---

### 3. The Go modules (bounded contexts)

Every module follows the same internal shape. Taking `bookings` as the archetype:

| File                        | Layer     | Responsibility                                                               |
|-----------------------------|-----------|------------------------------------------------------------------------------|
| <code>handlers.go</code>    | transport | parse/validate HTTP, RBAC<br>per route, call service, render<br>JSON/problem |
| <code>service.go</code>     | domain    | business rules, transactions,<br>state machine, conflict logic               |
| <code>state.go</code>       | domain    | the booking state-machine<br>transition table                                |
| <code>dto.go</code>         | transport | request/response shapes +<br>mappers from DB rows to<br>views                |
| <code>maintenance.go</code> | domain    | maintenance-window service<br>(booking-adjacent occupancy)                   |

| File      | Layer | Responsibility                                                       |
|-----------|-------|----------------------------------------------------------------------|
| *_test.go | —     | unit (state machine) +<br>integration (concurrency race,<br>BR1–BR6) |

Module-by-module:

- **iam** — authentication and identity. Login (Argon2id verify, logout, MFA), refresh-token rotation with reuse detection, password reset, TOTP MFA enrol/verify, and user/department management. Owns the `users`, `refresh_tokens`, `password_reset_tokens`, `departments` tables. Exposes `/api/v1/auth/*`, `/users`, `/departments`.
- **catalogue** — the physical estate: `buildings`, `equipment`, `rooms`, `room_equipment`. Also owns the **dynamic room search** (parameterised SQL with optional building/capacity/type/equipment filters) reused by the availability engine. Exposes `/buildings`, `/equipment`, `/rooms`.
- **scheduling** — `semesters` (with the single-ACTIVE-semester rule), `lecture_timetable_events`, and **ingestion** (CSV/XLSX → validate each row → insert, collecting per-row errors). Exposes `/semesters`, `/timetable/*`.
- **availability** — a **read-only engine**, the technical core. `intervals.go` is pure interval arithmetic (merge/subtract/overlap on minutes-of-day, 100% tested); `engine.go` gathers occupancy (active-semester lectures + approved bookings + maintenance) and subtracts it from the requested window; `calendar.go` builds the unified day/week/month block view. Exposes `/availability/search`, `/calendar`. Depends on `catalogue` (for candidate rooms) and reads `scheduling/booking/maintenance` data via the shared store.
- **bookings** — the booking request lifecycle (state machine PENDING → APPROVED / REJECTED / CANCELLED / EXPIRED), submit-time conflict detection (reject if it overlaps a lecture or maintenance), and the **concurrency-safe approval** (per-room advisory lock + partial EXCLUDE constraint → exactly one winner). Also admin override (BR6) and maintenance windows. Exposes `/bookings/*`, `/maintenance-windows`.
- **notifications** — channel-agnostic dispatch. Persists in-app notifications (durable), pushes them live over **SSE** via an in-process `Broker`, and sends email + push off the request path. Implements `bookings.Notifier` and `iam.Mailer`. Exposes `/notifications/*`, `/devices`.
- **reporting** — utilisation (lecture vs booked hours, expanding recurring lectures across a date range), booking stats, and conflict counts, with CSV export. Exposes `/reports/*`.

## Module dependency graph

```
flowchart TD
    subgraph transport [cmd/api — composition root]
        R[chi router + middleware]
    end
    R --> IAM[iam]
    R --> CAT[catalogue]
    R --> SCH[scheduling]
    R --> AV[availability]
    R --> BK[bookings]
    R --> NO[notifications]
    R --> REP[reporting]

    AV --> CAT
    BK -. Notifier .-> NO
    IAM -. Mailer .-> NO

    IAM & CAT & SCH & AV & BK & NO & REP --> PLAT[internal/platform/*]
    PLAT --> PG[(PostgreSQL 18)]
```

Cross-module coupling is deliberately tiny: `availability` reads catalogue rooms; `bookings` and `iam` notify via the `Notifier/Mailer interfaces`. Everything else goes through the shared platform store. No module imports another module's tables.

---

## 4. The platform layer (internal/platform)

Infrastructure shared by all modules. The key one is db:

- `db.Store` wraps the `pgx` pool and embeds the `sqlc-generated *dbgen.Queries`, so any module can run a typed query (`store.GetRoom(ctx, id)`).
- `Store.WithinTx / WithinTxDefault` run a closure in a transaction with a tx-bound `*dbgen.Queries` — used by the approval path and ingestion.
- `Store.Read / ReplicaPool` route read-only queries (availability, reporting) to a read replica when `DATABASE_REPLICA_URL` is set, else the primary.
- `MapError` translates Postgres errors (exclusion violations, the validation trigger's `RAISE`) into stable `apperr` codes — this is how the database's guarantees surface as clean API errors.

The rest:

### Package

`config`

`auth`

### Provides

typed env config (`config.Load()`), 12-factor; honours `$PORT` (`Render`)

| Package | Provides                                                                                                                                                                                                    |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         | HashPassword/VerifyPassword (Argon2id), TokenSigner (JWT), opaque refresh tokens, AESGCM, request Identity in context                                                                                       |
| rbac    | the §9.4 permission matrix; Can(role, permission), deny-by-default                                                                                                                                          |
| httpx   | RFC 9457 problem+json, JSON decode/encode, cursor pagination, and middleware: recover, correlation-id, access log, <b>Authenticator, RequirePermission, CSRF, RateLimit, SecurityHeaders</b> , auth cookies |
| apperr  | the catalogue of typed domain errors (status + stable code + fields)                                                                                                                                        |
| audit   | append-only audit-log recorder (every state change)                                                                                                                                                         |
| mailer  | Mailer interface + log (dev) and SMTP implementations                                                                                                                                                       |
| metrics | Prometheus counters/histograms (bookings created/approved/..., search latency)                                                                                                                              |
| logging | slog JSON + correlation id helpers                                                                                                                                                                          |
| pgconv  | Go time <a href="#">?</a> pgtype conversions                                                                                                                                                                |

**Data access generation:** you write SQL in `db/queries/*.sql` with `sqlc` annotations; `sqlc generate` (config in `sqlc.yaml`) reads the schema from `db/migrations/` and emits type-safe Go into `internal/platform/db/dbgen/`. CI fails if the committed generated code drifts. **Never hand-edit** `dbgen/`.

---

## 5. Request lifecycle (how to read a request)

Every authenticated API call flows through the same middleware chain, assembled in `cmd/api/buildRouter`:

```
sequenceDiagram
```

```

    participant C as Client (web/mobile)
    participant MW as chi middleware
    participant H as module handler
    participant S as module service
    participant DB as PostgreSQL

```

```
C->>MW: HTTP request (Bearer token or __Host-/cbs- cookie)
```

```
MW->>MW: RealIP → Correlation → Recover → AccessLog → CORS → SecurityHea
```

```
MW->>MW: Authenticator (verify JWT → Identity in ctx) → CSRF → RequirePe
```

```
MW->>H: handler(w, r)
```

```

H->>H: decode + validate body (httpx.DecodeJSON)
H->>S: service call (domain logic)
S->>DB: typed query / transaction (dbgen + Store)
DB-->>S: rows / constraint error
S-->>H: domain value or apperr (via db.MapError)
H-->>C: JSON 2xx OR application/problem+json (RFC 9457, stable `code`)

```

Auth specifics: web uses httpOnly cookies (\_\_\_Host-access/\_\_\_Host-refresh over HTTPS; cbs\_access/cbs\_refresh over local HTTP) plus a double-submit X-CSRF-Token; mobile uses Authorization: Bearer. RBAC is centralised — handlers declare a permission via `httpx.RequirePermission(...)`; object-level ownership (e.g. a requester reading only their own booking) is checked inside the handler.

---

## 6. The two occupancy datasets and how availability is derived

This is the heart of the domain. There is **no “is this room free” flag** anywhere — availability is computed on demand.

```

flowchart LR
    subgraph Inputs
        L[timetable_events<br/>recurring lectures<br/>active semester only]
        B[bookings<br/>status = APPROVED only]
        M[maintenance_windows]
    end
    L --> E{availability engine<br/>interval subtraction}
    B --> E
    M --> E
    Q[query: date + window + filters] --> E
    E --> FREE[rooms free for the whole window<br/>+ their free sub-interval]

```

- A room is **occupied** at instant  $t$  on date  $D$  iff an active-semester lecture for weekday ( $D$ ) covers  $t$ , **or** an APPROVED booking on  $D$  covers  $t$ , **or** a maintenance window covers  $t$ .
  - The engine clamps everything to local minutes-of-day and uses `availability/interval.go` (Subtract, Merge, Overlaps) — half-open intervals, so a lecture ending at 10:00 leaves the room free *from* 10:00.
-

## 7. The double-booking guarantee and the approval race

The database — not the application — is the source of truth for “is this slot taken”:

- A **partial exclusion constraint** (`excl_booking_overlap`) makes two *APPROVED* bookings for the same room with overlapping time **impossible**. PENDING requests do not reserve (so many can compete for one slot).
- A **validation trigger** (`fn_validate_booking`) independently enforces not-in-past, single-day, capacity, and (when becoming APPROVED) lecture/maintenance precedence.
- On approval, `bookings.Service.Approve` takes a **per-room advisory lock**, flips the status inside a transaction, and lets the constraint settle the race; a violation is mapped to 409 `SLOT_NO_LONGER_AVAILABLE`. The mandatory concurrency test proves: N officers approve competing requests → exactly one wins, the rest 409, DB ends with one approved row.

stateDiagram-v2

```
[*] --> PENDING: requester submits (lecture/maintenance conflicts reject)
PENDING --> APPROVED: officer approves [advisory lock + EXCLUDE]
PENDING --> REJECTED: officer rejects (note required)
PENDING --> CANCELLED: requester/officer cancels
PENDING --> EXPIRED: worker sweep (start time passed)
APPROVED --> CANCELLED: cancel / admin override (BR6)
```

---

## 8. The web app (apps/web)

Next.js 16 App Router, organised by **route groups** (URL-invisible folders that share a layout) and **feature folders**:

```
src/app/
├── (public)/          landing + room directory + privacy/terms (Ghana Act
├── (auth)/            login, forgot-password, reset-password
├── app/              REQUESTER portal (gated): /app, /app/search, /app/
├── admin/            ADMIN console (gated): dashboard, approvals, ro
├──                   semesters, timetable (alias-tolerant import + room
└── layout.tsx        root: no session read (public stays static for SEO)
```

- `src/proxy.ts` (Next 16’s renamed middleware) is the cheap first gate: it redirects `/app/*` and `/admin/*` to `/login` if the access cookie is absent, and stamps X-Robots-Tag: noindex. The **server layouts** (`app/layout.tsx`, `admin/layout.tsx`) do the authoritative session + role gate via `getSession()`.
- Data: Server Components fetch via the typed client; interactive bits are Client Components using `@tanstack/react-query`. Forms use `react-hook-form` + the shared zod schemas. The notifications bell subscribes to the SSE stream.

- The app talks to the API through `next.config.ts` rewrites (`/api/v1/*` → `API_ORIGIN`), so calls are same-origin (cookies + CSRF work without CORS) — the same proxy makes the Vercel→Render split possible.
- **Theming:** light/dark plus selectable dark “tints” — a `data-dark-tint` attribute drives CSS-variable overrides in `@cbs/ui`; a blocking inline script applies the saved mode + tint before first paint (no FOUCC) and `ThemePreferenceSync` keeps it live. Account **Settings** is a tabbed surface (Profile / Security / Notifications / Preferences) carrying the tint picker.

## Shared TS packages

- `@cbs/api-client` — `createApi({ fetch })` (openapi-fetch) with CSRF middleware, `ApiError/unwrap` helpers, and `schema.gen.ts` generated from `api/openapi.yaml`.
  - `@cbs/schemas` — zod schemas + enums + the stable error-code list, shared by web forms and (a copy in) mobile.
  - `@cbs/tokens` — framework-neutral source of truth for the Ashesi brand palette and the dark-tint catalogue (value/label/swatches), consumed by both web (`@cbs/ui` + theme prefs) and mobile so the two stay in lockstep.
  - `@cbs/ui` — Tailwind v4 `@theme` tokens (incl. the `.dark[data-dark-tint=...]` palettes), shadcn-style components, `cn` and institution-timezone datetime helpers (`Africa/Accra`).
  - `@cbs/config` — shared eslint/tsconfig.
- 

## 9. The mobile app (apps/mobile)

Expo SDK 56 + expo-router (file-based) + NativeWind. Screen groups mirror roles: `(auth)/login`, `(requester)/(search → results → request with an availability view, bookings, booking/[id], notifications, settings)`, `(officer)/(approvals with blocker panels, reports, notifications, settings)`. It uses Bearer tokens in `expo-secure-store`, the same zod schemas (local copy), and registers an Expo push token via `POST /devices`. Theming mirrors web — light/dark + the same dark tints from `@cbs/tokens`, persisted via `AsyncStorage` and applied on launch, with the Outfit font. Local APK builds use an `eas.json` profile (`pnpm apk → apps/mobile/build/aura.apk`). It type-checks + lints; not yet run on a physical device.

---

## 10. Background work & deployment

- **Background jobs are folded into the API.** `cmd/api` runs the booking-expiry sweep (PENDING whose start has passed → EXPIRED) and idempotency-key cleanup on tickers in a goroutine (`runBackgroundJobs`), stopped on shutdown. `cmd/worker` still holds the same logic for a separate-worker / River deploy (ADR-0002), but the default single-service deploy needs no extra worker.

- **The API self-bootstraps on startup** so a shell-less, pre-deploy-less host just works: `AUTO_MIGRATE=true` applies the embedded goose migrations on boot (`runMigrations`), then `SEED_DATA=true` loads the embedded, idempotent `db/seed/seed.sql` (clean demo data) via the `pgx` simple-query protocol (`runSeed`). A `uuidv7()` polyfill (migration `00001`) keeps the schema portable to PG 16/17.
  - **Deploy is split:** the **web** app → **Vercel** (root `apps/web`); the **Go API + Postgres** → **Render** (`render.yaml` Blueprint, one Docker image). The browser only talks to the Vercel origin; Next rewrites `/api/v1/*` to the API so cookies stay first-party. To keep costs low there is no separate worker and no Redis (rate-limit + SSE are in-process); managed Blueprint Postgres needs a paid plan (`basic-256mb`), with a free-Postgres-via-dashboard alternative documented in `render.yaml`. Mail is **Resend** (SMTP); catalogue images are **Cloudinary**. The Helm chart + Terraform under `deploy/` remain valid for a Kubernetes target. See `docs/DEPLOYMENT.md`.
- 

## 11. Core algorithms

This section describes the important algorithms that drive the domain. They are implemented in the Go modules; the web/mobile apps mirror the rules for UX but the API is the source of truth.

### 11.1 Interval arithmetic (the availability primitive)

All time-window calculations work in **minutes-from-midnight** on a local (`Africa/Accra`) day using half-open intervals [`start`, `end`).

```
type Interval struct { Start, End int } // 0 ≤ Start ≤ End ≤ 1440
```

Half-open semantics matter: a lecture ending at 10:00 and a booking starting at 10:00 **do not overlap**, so the room is free at exactly 10:00. The primitives in `internal/availability/intervals.go` are:

- `Overlaps(a, b)` — true iff `a.Start < b.End && b.Start < a.End`.
- `Merge(intervals)` — sort by start, then coalesce overlapping or adjacent intervals into a minimal disjoint set.
- `Subtract(window, occupied)` — merge and clamp `occupied` to the window, then emit the free gaps before, between, and after occupied blocks.
- `FullyFree(window, occupied)` — true iff no occupied interval overlaps the requested window (used to reject rooms that are busy for any part of the slot).

These functions are pure and fully unit-tested; they are the only place interval math lives.

## 11.2 Availability search

GET /api/v1/availability/search answers “which rooms are free for my whole window, and what smaller free gaps do they have around it?”

1. Force the filter to `Status = ACTIVE` (only bookable rooms).
2. Call `catalogue.SearchRooms` for candidates.
3. For each room, call `dayOccupancy(roomID, date)`:
  - Load active-semester lectures for that weekday.
  - Load **APPROVED** bookings on that day.
  - Load maintenance windows on that day.
  - Convert lecture time columns and clamp booking/maintenance UTC instants to local minutes `[0, 1440]`.
4. Return Busy blocks (labelled **LECTURE/BOOKING/MAINTENANCE**) and `FreeIntervals = Subtract([0, 1440], occupied)`.

The requested start/end is a **client highlight**, not a server filter. This lets users see the full day and click any free gap, even one smaller or larger than their original search.

## 11.3 Calendar block generation

GET /api/v1/calendar builds the unified day/week/month view.

1. `rangeFor(view, date)` expands the query date into a local range:
  - `day` — the single date.
  - `week` — ISO Monday–Sunday.
  - `month` — full calendar month.
2. `scopeRooms` resolves to one room or all rooms in a building (capped at 200).
3. For every date × room:
  - Load lectures, **approved and pending** bookings, and maintenance.
  - Emit a labelled block for every source.
  - Add only **APPROVED** bookings to the occupancy set; pending bookings are visible but do not reduce availability.
  - Emit **AVAILABLE** gaps with `Subtract([0, 1440], occupied)`.

This is why the booking dialog’s pre-submit availability check must ignore **PENDING** booking blocks even though they appear on the calendar.

## 11.4 Booking a room (submit)

POST /api/v1/bookings creates a **PENDING** request.

1. Validate `purpose`, `attendee_count` in `[1, 100000]`, and `ends_at > starts_at`.
2. Run `hardConflicts` against **lectures and maintenance only**:
  - Build the request interval in local minutes.
  - Load the room’s lectures for that weekday and maintenance for that day.

- Reject with `CONFLICTS_WITH_LECTURE` or `CONFLICTS_WITH_MAINTENANCE` on any overlap (BR1).
- 3. Insert the row as `PENDING`.
  - The `fn_validate_booking` trigger independently rejects:
    - past times (`BOOKING_IN_PAST`);
    - multi-day spans (`BOOKING_SPANS_MULTIPLE_DAYS`);
    - capacity exceeded (`ATTENDEES_EXCEED_CAPACITY`).
- 4. Count competing `PENDING` overlaps (excluding the new row) and return `SubmitResult{Booking, CompetingPending}` so the UI can warn the requester that others want the same slot (FR8).

Approved bookings are **not** checked at submit time; the database resolves that race at approval time.

## 11.5 Booking approval and the double-booking race

`PATCH /api/v1/bookings/{id}/approve` moves a booking to `APPROVED`.

1. `FOR UPDATE` lock the booking row and validate the state transition.
2. Acquire a **per-room PostgreSQL advisory lock** keyed on the room ID string. This serialises concurrent approval attempts for the same room.
3. Update status to `APPROVED` inside a transaction.
4. The partial exclusion constraint `excl_booking_overlap(room_id WITH =, during WITH && WHERE status = 'APPROVED')` is the ultimate race resolver. If another officer approved a conflicting booking first, this transaction gets a constraint violation, mapped to `409 SLOT_NO_LONGER_AVAILABLE`.
5. Commit and notify.

A concurrency test proves that  $N$  officers approving competing requests for the same slot result in exactly one `APPROVED` booking and  $N-1$  `409` responses.

## 11.6 Booking state machine

`internal/bookings/state.go` encodes the transition table:

```
PENDING → APPROVED | REJECTED | CANCELLED | EXPIRED
APPROVED → CANCELLED
REJECTED, CANCELLED, EXPIRED are terminal
```

Every transition is validated before any DB update. Terminal states have no outgoing transitions, so a rejected or cancelled booking cannot be resurrected.

## 11.7 Maintenance windows

`POST /api/v1/maintenance-windows` inserts an occupied window.

1. Validate `ends_at > starts_at`.
2. Insert the row.

3. The exclusion constraint `excl_maint_overlap` prevents two maintenance windows for the same room from overlapping.
4. The symmetric trigger `fn_validate_maintenance` also blocks a maintenance window from being created on top of an existing APPROVED booking.

Maintenance windows are treated exactly like lectures in availability and approval calculations.

## 11.8 Room search and filtering

`GET /api/v1/rooms` builds dynamic SQL with parameter binding only (no string interpolation of values).

1. Join `rooms` → `buildings`.
2. Apply optional filters: `status, building_id, capacity >= min, room_type, name ILIKE '%query%'`.
3. For each equipment term, add an EXISTS subquery matching `lower(e.code) = term OR lower(e.name) = term`.
4. Cursor pagination on `r.id > cursor`, ordered by `r.id`.
5. Enforce limit bounds (default/capped).

The same `RoomFilter` is reused by the availability engine.

## 11.9 Timetable import / ingestion

`POST /api/v1/timetable/import` ingests CSV or XLSX timetable events for a semester.

1. Parse the file with a 64 MB unzip/size limit and a 20,000-row cap.
2. `mapHeaders` resolves canonical columns via case-insensitive aliases (`course code`, `course name`, `staff name`, `location`, `day`, `from time`, `to time`, etc.).
3. `ParseClock` normalises `08:00`, `8:00 AM`, `0800`, `1430`, etc. to 24-hour (`hour`, `min`).
4. `ParseDay` maps `MON/MONDAY/three-letter` forms to `day_of_week`.
5. In `replace` mode, delete all existing events for the semester first (bookings are untouched).
6. For each row:
  - Resolve the room by code, then by name; if `createMissing` is true, provision a building + room on the fly.
  - Deduplicate within the upload using a signature `room|day|start|end`.
  - Validate `end > start` and semester bounds.
  - Insert valid rows; collect invalid rows as `RowError{row, field, message}`.
7. Persist the final status (`COMPLETED`, `PARTIALLY_COMPLETED`, or `FAILED`) and an `error_report` JSONB.

The one-active-semester rule (`uq_one_active_semester`) and the lecture overlap exclusion (`excl_tt_overlap`) are enforced by the database.

## 11.10 Reporting / utilisation

`GET /api/v1/reports/utilisation` computes room usage over a date range.

1. CTE `booked`: sum approved booking durations in hours.
2. CTE `lectures`: sum active-semester timetable-event durations multiplied by the number of matching weekdays inside the semester date range.
3. Join rooms; compute:
  - `used = lecture_hours + booked_hours`
  - `available = workingHoursPerDay × days_in_range - used`
  - `utilisation_pct = used / available × 100`
4. Return per-room rows plus an average.

Other reports (`/reports/bookings`, `/reports/conflicts`, `/reports/overview`) use similar SQL CTEs and `generate_series` spines for daily/hourly series.

## 11.11 Authentication and session refresh

`POST /api/v1/auth/login`:

1. Look up user by email. If missing, verify against a precomputed dummy Argon2 hash to equalise timing (no user enumeration).
2. Check account lock (`locked_until`).
3. Verify password with Argon2id.
4. Reject `SUSPENDED` users.
5. If MFA is enabled, require and verify a TOTP code (failures count as login failures).
6. Issue tokens.

`issueTokens`:

1. Sign a short-lived access JWT (user ID, role, email).
2. Generate an opaque refresh token, hash it, and store it with a `family_id`, user-agent, IP, and expiry.

`POST /api/v1/auth/refresh`:

1. Look up the token by hash.
2. If already revoked, revoke the **whole family** and return `ErrTokenReuse`.
3. If expired or user suspended, reject.
4. Revoke the current token (rotation) and issue a new pair in the same family.

Password changes, role changes, and suspension revoke all refresh tokens for the user, forcing re-login.

---

## 12. How to navigate / extend, by task

### You want to...

Change an API behaviour

Add an endpoint

Change the schema

Understand availability

Understand the booking guarantee

Change permissions

Change error codes/shape

Add a web page

Trace a request

See decisions/trade-offs

...

### Start here

the module's `service.go` (logic) +  
`handlers.go` (HTTP)

`handlers.go` route + `service.go`  
method + a `db/queries/*.sql` → `sqlc`  
`generate`

`new db/migrations/NNN_*.sql`  
(forward-only) → `sqlc generate`

`internal/availability/`  
`intervals.go` then `engine.go`

migration `00005` (constraints + trigger) +  
`bookings/service.go:Approve`

`internal/platform/rbac/rbac.go`

`internal/platform/apperr` +  
`httpx/respond.go`

`apps/web/src/app/...` (pick the right  
route group)

`cmd/api/buildRouter` → middleware  
→ module handler → service → dbgen  
query

`docs/decisions/` (ADRs)